HW5_alex_tiu

October 21, 2026

1 Problem 5.1

[1]:
```python
# Write the bisection method using the pseudocode

# Let a and b represent the endpoints. Let f be the function we are evaluating␣
 ↪the roots for
def bisection_method(f, a, b, tolerance):

    # Counter
    k = 0

    # Plug in a and b into f
    fa = f(a)
    fb = f(b)

    # Get the midpoint c from a and b
    c = (a + b) / 2

    # If fa and fb have the same sign, the interval does not bracket a root.
    # We need an interval whose endpoint values have opposite signs (IVT).
    if (fa * fb > 0):
        raise ValueError(f"f(a)={fa} and f(b)={fb} must have different signs!")

    # While b - a stays above the tolerance
    while (b - a > tolerance):

        # Get the midpoint again
        c = (a + b) / 2

        # Find the value of the function at c
        fc = f(c)

        # If fc equals 0, then we have found the root!
        if fc == 0:
            return c

        # If there is a sign change in [a, c], keep that interval.
```

```
        elif fa * fc < 0:
            b = c
            fb = fc

        # Otherwise, the sign change is in [c, b].
        else:
            a = c
            fa = fc

        # Counter for number of iterations
        k += 1

    return (a + b) / 2, k
```

[2]:
```
# Tests
# Use 10 + x^3 - 12cos(x) = 0
import numpy as np

def f(x):
    return 10 + x**3 - 12 * np.cos(x)

e = 1e-4
a = -5
b = 5

root, iterations = bisection_method(f, a, b, e)

print(f"Let the interval be [{a},{b}]")
print(f"Let the tolerance e be: {e}")
print(f"For this interval to be a good interval to start with, one of the␣
 ↪values should be negative")
print(f"f({a}) = {f(a)}")
print(f"f({b}) = {f(b)}")
print(f"Because one of them is negative, we can continue\n")
print("-" * 15)
print(f"Number of iterations: {iterations}")
print(f"Root is at: {root}")
print(f"f({root}) = {f(root)}")
```

```
Let the interval be [-5,5]
Let the tolerance e be: 0.0001
For this interval to be a good interval to start with, one of the values should
be negative
f(-5) = -118.40394622555871
f(5) = 131.5960537744413
Because one of them is negative, we can continue

---------------
```

```
Number of iterations: 17
Root is at: 0.5588150024414062
f(0.5588150024414062) = -0.00010413245379048419
```

2 Problem 5.2

[ ]:
```python
# Create Newton's Method
# Let r = the final approximation
# Let f = the name of a nonlinear residual function
# Let df = the derivative of f
# Let x0 = the initial guess
# Let nmax = the number of allowed iterations
# Let etol = error tolerance
# Let ishow = flag to enable/disable printing of iteration information

# Define lambda functions for f and df
# x is the argument
f = lambda x: (x**2) * (x - 3) * (x + 1)
df = lambda x: 4 * (x**3) - 6 * (x**2) - 6 * x

nmax = 100
x0 = [-3, 1, 2, 4]
etol = [1e-6, 1e-10]

# Newton's Method
def newton_method(f, df, x0, nmax, etol, ishow):

    # Let the value of x0 become r
    r = x0
    for k in range(1, nmax + 1):
        estimate = f(r) / df(r)

        # Get the next approximation
        r = r - estimate

        if (ishow):
            print(f"Iteration: {k}, Approximation: {r}")

        # If the absolute error is less than the tolerance, we are finished
        if abs(estimate) < etol:
            if (ishow):
                print(f"Iteration: {k}, Approximation: {r}, finished")
            return r

    # Return the last approximation if nmax is reached
    return r
```

```python
# Main
ishow = True

for x in x0:

    for tol in etol:

        r = newton_method(f, df, x, nmax, tol, ishow)
        print(f"Initial guess: {x}")
        print(f"Tolerance: {tol}")
        print(f"Root is at: {r}")
        print(f"f({r}) = {f(r)}")
        print("-" * 15)
```

```
Iteration: 9, Approximation: -1.0, finished
Initial guess: -3
Tolerance: 1e-06
Root is at: -1.0
f(-1.0) = -0.0
---------------
Iteration: 9, Approximation: -1.0, finished
Initial guess: -3
Tolerance: 1e-10
Root is at: -1.0
f(-1.0) = -0.0
---------------
Iteration: 21, Approximation: 5.777117764399681e-07, finished
Initial guess: 1
Tolerance: 1e-06
Root is at: 5.777117764399681e-07
f(5.777117764399681e-07) = -1.0012530755358064e-12
---------------
Iteration: 34, Approximation: 7.05214842423451e-11, finished
Initial guess: 1
Tolerance: 1e-10
Root is at: 7.05214842423451e-11
f(7.05214842423451e-11) = -1.4919839219931432e-20
---------------
Iteration: 2, Approximation: -1.0, finished
Initial guess: 2
Tolerance: 1e-06
Root is at: -1.0
f(-1.0) = -0.0
---------------
Iteration: 2, Approximation: -1.0, finished
Initial guess: 2
Tolerance: 1e-10
Root is at: -1.0
```

```
f(-1.0) = -0.0
---------------
Iteration: 6, Approximation: 3.0, finished
Initial guess: 4
Tolerance: 1e-06
Root is at: 3.0
f(3.0) = 0.0
---------------
Iteration: 7, Approximation: 3.0, finished
Initial guess: 4
Tolerance: 1e-10
Root is at: 3.0
f(3.0) = 0.0
---------------
```

2.0.1 Part 1

How does the performance of Newton's method change as you vary the initial guess?

As the initial guess varies, Newton's method may converge to different roots. It does not always converge to the closest root. For example, $x_0 = 2$ converges to -1, even though 3 is the closest root.

2.0.2 Part 2

How does Newton's method change as you vary the tolerance?

As the tolerance gets smaller, Newton's method usually needs more iterations because the approximation must be more accurate. For example, when $x_0 = 1$, a tolerance of 10^{-6} takes 21 iterations, while 10^{-10} takes 34 iterations.

2.0.3 Part 3

Explain your results.

The approximations converge to the roots -1, 0, and 3. The initial guesses -3 and 2 converge to -1, 1 converges to 0, and 4 converges to 3. The method takes more iterations when $x_0 = 1$ because 0 is a repeated root, so Newton's method converges more slowly.